Machine Learning Course Project-I

Flight Price Prediction (Regression)

Team Name: Carrot

Name	Roll number	Contribution
Aradhya Chakrabarti	MT2026012	Preprocessing for the flight competition (data loading, cleaning and feature engineering); Models Tested: linear regression (OLS), polynomial regression, Ridge, Lasso, decision trees, random forests (with cross-validation, evaluation and hyperparameter tuning)
Ankit Lamba	MT2026122	Exploratory testing and data analysis for the flight competition; tested SVR, linear regression and random forest baselines
Kunal Yadav	MT2026047	Ran preliminary prediction experiments for the flight competition
Kshitij Beri	MT2026046	Implemented an ensemble model (blend) of trees, forests and boosting; achieved the team's best public leaderboard score on the flight competition

Source code

The full code for this competition is in the project repository: `https://github.com/TimeATronics/AIT511_Project1`

1 Problem and dataset

We are given a regression problem: predict the ticket price (in USD) of a US domestic flight. The training file `train.csv` has 18,076 rows and 12 columns, and the test file has 4,520 rows and 11 columns. The columns are airline, flight, source_city, destination_city, departure_time, arrival_time, stops, flight_class, duration (hours), days_left, and the target price. The evaluation metric is R^2.

We have used the following models: linear regression (ordinary least squares), polynomial regression, Ridge (L2), Lasso (L1), decision trees, and random forests, along with various experimentations and combinations of the same.

2 Data loading

We begin by loading the three CSV files (training, test and the sample submission) with pandas. Both files turn out to be complete: there are no duplicate rows and no missing cells in either file, so no imputation is needed in practice. We still keep a median/mode imputer inside the pipeline so that the code would not break if missing values appeared later.

3 Exploratory data analysis

3.1 Data distribution check (skewness)

For EDA, the first thing we check is the shape of the target, since it decides which error metric is sensible and whether a transformation could help the linear models. We compute the descriptive statistics along with skewness and kurtosis, and plot the raw prices next to $\log(1 + \text{price})$ (Figure 1).

From the histogram we see that most tickets are concentrated below USD 400, while a handful of fares reach USD 8,996 against a median of only USD 234. The computed skewness (7.07) and kurtosis (170.25) quantify how strongly these few expensive tickets pull the distribution away from the typical fare. Since the models are scored with squared error, these few tickets can dominate the score.

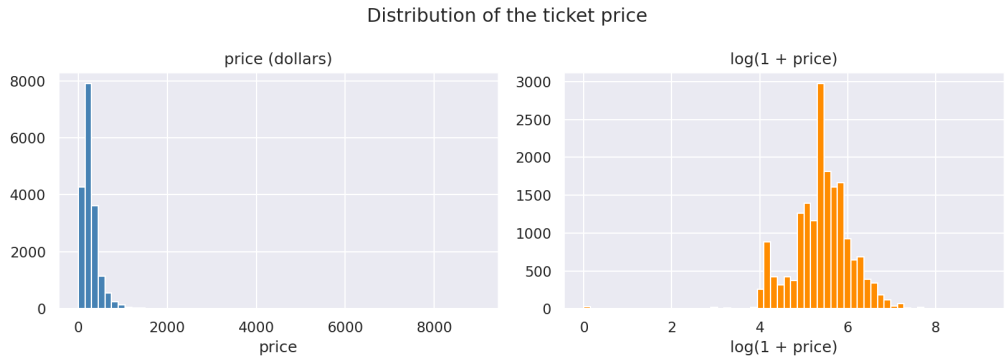


Figure 1: Price distribution in raw dollars and after $\log(1 + \text{price})$.

3.2 Feature-wise analysis

Next we look at the two numeric features and at how price behaves across the categorical ones. The code prints the descriptive statistics of `duration` and `days_left`, their correlations with price, and draws boxplots for the numeric columns and for price across seating class, stops and airline (Figure 2). Accordingly, we observe the following:

- Business tickets are clearly higher in price, with median near USD 320 against USD 194 for Economy. Business class has the outlier expensive fares near USD 8,996.
- Non-stop flights have a median near USD 214, while one-stop and two-or-more itineraries are around USD 347 and USD 370. Only 193 flights have two or more stops, so that group is relatively small.
- Airline medians range from about USD 168 for Frontier up to about USD 260 for United, Delta and Alaska. Southern Airways Express appears exactly once in the training data, so nothing general can be learned for it.
- `duration` ranges from 0.87 to 28.65 hours, though a few flights run much longer than the rest, and `days_left` covers 1 to 44 days. Neither relates strongly to price since the linear correlations are only $+0.281$ and -0.229.

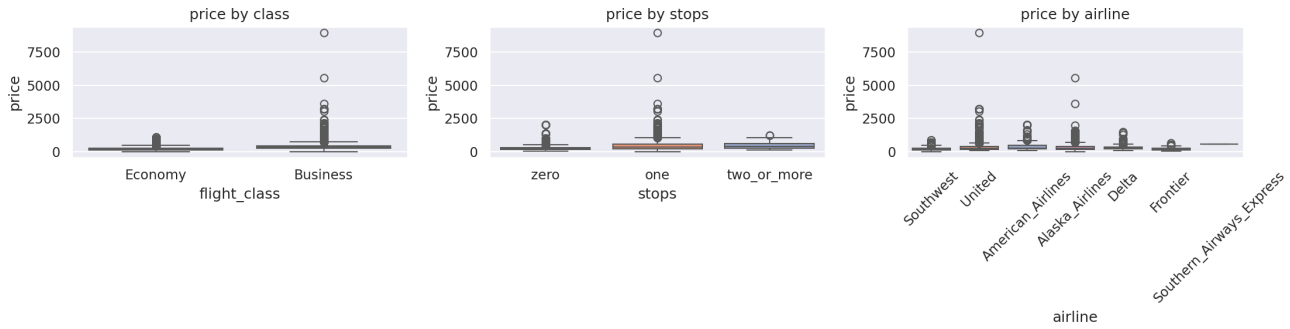


Figure 2: Price by seating class, number of stops and airline.

3.3 Correlation analysis

We then check the correlations between the numeric columns. The heatmap block shows that no pair is strongly linearly related, and the largest correlation with price is $+0.28$ for `duration`. A linear model trained only on the numeric columns would probably perform not as well as models considering the categorical columns as well. See Section 8.

3.4 High-cardinality columns

Finally, we inspect the categorical cardinalities. `flight` has 1,123 distinct codes, and 33 of them appear only in the test file. Treating it as a category would create over a thousand sparse columns and the unseen codes would have no coefficients at all, so we replace it with engineered features in Section 5.

4 Data cleaning

For cleaning, we find 25 training rows with a price of 0. Since a ticket cannot cost nothing, these look like entry errors and we remove them; 18,051 rows remain for training.

5 Feature engineering

We build three features from existing columns:

1. `airline_code` and `flight_number` from the flight code. For example, the code "WN 3596" contains the carrier ("WN") and a serial number, and airlines tend to reuse numbers for the same route, so the number can carry route information that the city columns would not convey.
2. `stops_ordinal`: `stops` is one-hot encoded during preprocessing for the regression models.
3. `route`: source and destination are merged into one category. The same city pair have the same fare level this way.

6 Data preprocessing

All preprocessing is done with a scikit-learn `ColumnTransformer` and used inside a `Pipeline`, so during cross-validation (CV) every transformation is fitted on the training folds only and the not the validation folds.

- Numeric columns are imputed with the column median and standardised for the models that need it. Scaling is done for the regression models with penalization (Ridge and Lasso). Trees split on thresholds and use raw values.
- Categorical columns are imputed with the most frequent value and one-hot encoded, with the `handle_unknown` option set to `"ignore"`.
- Columns with more than 100 levels are dropped by the encoder; `flight` is the only such column.

7 Model evaluation

For targets y_i and predictions $\hat{y}_i = f(x_i; w)$, we report

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \qquad \text{RMSE} = \sqrt{\text{MSE}}, \qquad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|,$$

and

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

Minimising the squared error on the training folds is the objective of OLS, Ridge and Lasso (Section 9), while the reported RMSE is computed on the remaining fold.

For validation we split the training data into $K = 5$ folds, hold each fold out once while the model trains on the remaining $K - 1$ folds, and average the five scores: $\frac{1}{K}\sum_{k=1}^{K} e_k$. The finalists are re-checked with 5 folds repeated 3 times (15 scores each). A fixed random state (42) is used for every fold split for reproducibility.

8 Model family direct comparisons (baseline)

We first train each model family once with reasonable settings. The families are: ordinary least squares (OLS), polynomial regression (with/without ridge, lasso), decision trees, and random forest. See: Figure 3.

Model	CV RMSE
Random forest (300, sqrt)	128.97
Ridge (L2, $\lambda = 1$)	153.64
Lasso (L1, $\lambda = 0.1$)	154.30
Linear regression (OLS)	154.82
Decision tree (unrestricted)	158.25
Polynomial regression (degree 2)	177.25

Table 1: Baseline comparison of the model families.

The random forest is already better than the rest (around 128 RMSE against about 154 or higher for the other families). Polynomial regression without a penalty is worse than plain linear regression. The unrestricted tree overfits, scoring about 158, close to the linear models.

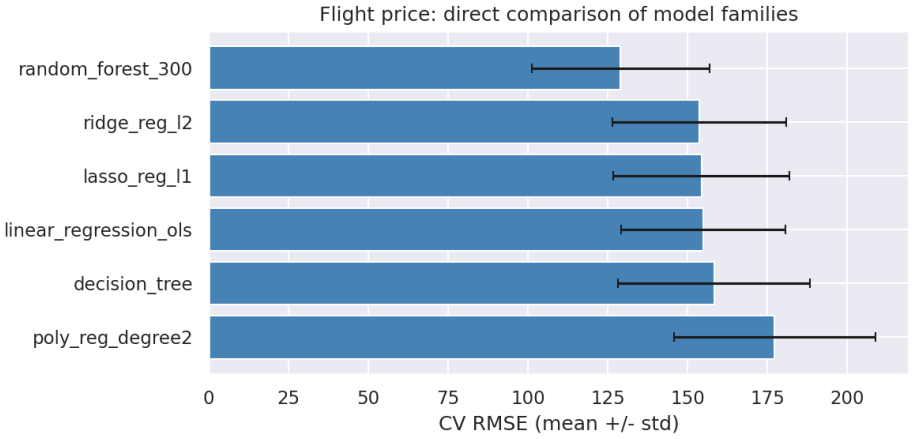


Figure 3: Direct comparison of the model families (CV RMSE).

9 Hyper-parameter tuning

9.1 Polynomial regression: optimal degree

Since the polynomial expansion is the natural way to give a linear model curvature, we tune its degree first. The degree controls how much curvature and interaction the model can represent: degree 1 is a straight line, degree 2 adds squares and pairwise products, and degree 3 adds triple products. We loop over degrees 1 to 3, fit OLS and Ridge ($\lambda = 1$) on each expansion and plot the curves (Figure 4).

Polynomial degree	CV RMSE, L2 ($\lambda = 1$)	CV RMSE, no penalty
1	153.6	154.8
2	136.7	177.3
3	158.0	380.9

Table 2: Polynomial regression: cross-validated RMSE by degree.

Degree 2 without a penalty is worse than degree 1 at 177.3 RMSE, and degree 3 goes up to 380.9 RMSE. With this much flexibility the model fits individual training rows, including noise and hence has high validation error. The same degree-2 expansion with an L2 penalty gives 136.7 RMSE, which is the best linear model we have seen so far. Hence we choose this for further experimentation with Ridge and Lasso regularization.

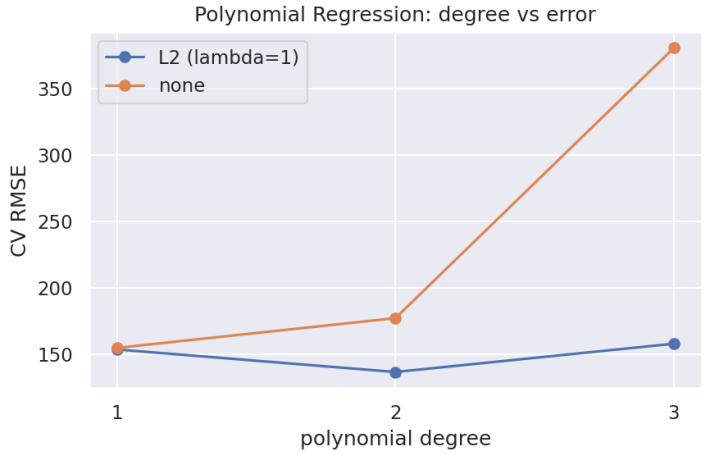


Figure 4: Polynomial degree against Cross Validation (CV) error, with and without an L2 penalty.

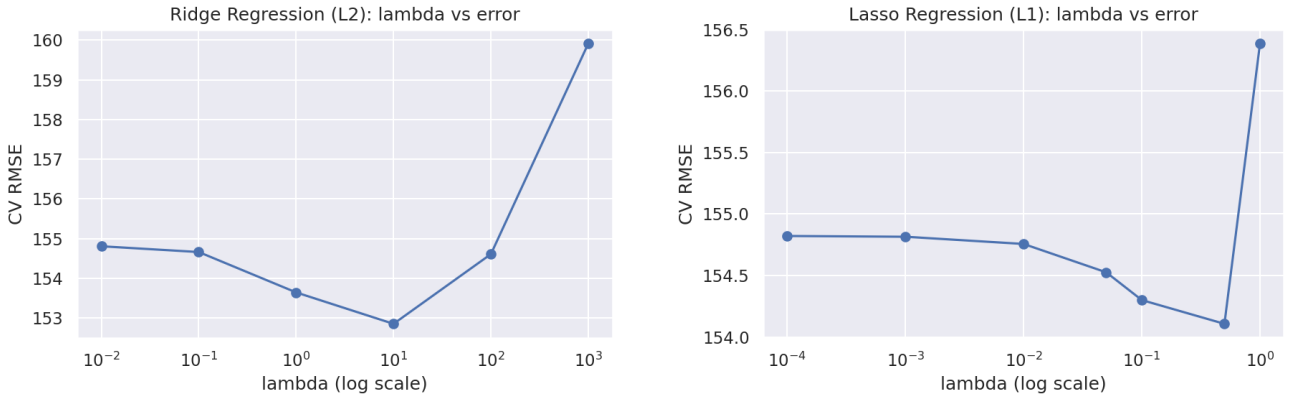
9.2 Ridge and Lasso regularisation

Ridge minimises the squared error plus $\lambda\|w\|_2^2$. A larger λ shrinks the coefficients more. We iterate over $\lambda \in \{0.01, \ldots, 1000\}$ and plot the curve (Figure 5, left). The curve is flat between $\lambda = 1$ and 100 (152.9 to 154.6 RMSE), with the best value at $\lambda = 10$ (152.9).

Lasso minimises the squared error plus $\lambda\|w\|_1$. For coefficient w_j the optimality condition is

$$\frac{2}{n}\,\mathbf{x}_j^\top(\mathbf{y} - \mathbf{X}w) = \lambda\, s_j, \qquad s_j \in \partial|w_j|,$$

so a coefficient is set to zero whenever $\left|\frac{2}{n}\mathbf{x}_j^\top(\mathbf{y} - \mathbf{X}w)\right| \le \lambda$. We iterate over λ from 10^{-4} to 1, refit the best value and count the remaining features. Lasso has the best result at $\lambda = 0.5$ with 154.1 RMSE, slightly behind Ridge, with 42 non-zero features out of 63 encoded features.



(a) Ridge **(b)** Lasso

Figure 5: Regularisation curves.

9.3 Decision tree: pre-pruning and post-pruning

An unrestricted tree grows until its leaves are maximally pure. Hence to restrict overfitting, we carry out pre-pruning, for which we tune four limits together: maximum depth, minimum samples per leaf, minimum samples needed to split a node, and the maximum number of leaves. We observe best performance with depth 12 with a minimum leaf size of 5, having 128.8 RMSE.

For post-pruning we grow the tree fully and cut it back with the following cost-complexity criterion

$$R_\lambda(T) = R(T) + \lambda|T|,$$

where $R(T)$ is the training error and $|T|$ the number of leaves, so λ is the penalty per leaf. Pruning back to 66 leaves ($\lambda = 66.2$) gives about 148 RMSE, worse than the pre-pruned tree at 128.8, hence we keep the pre-pruned tree (Figure 6).

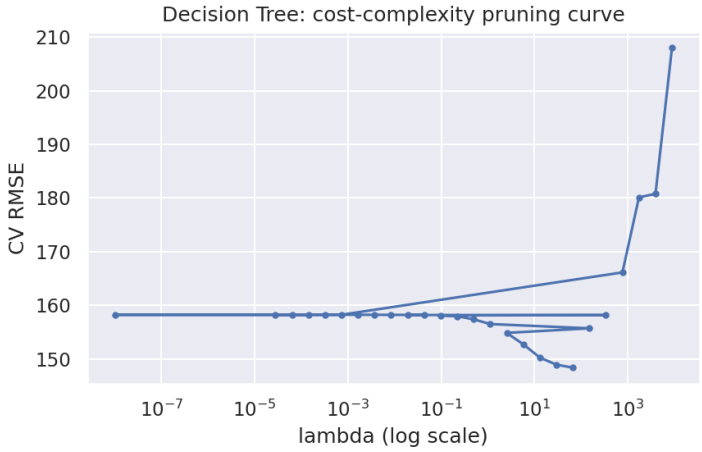


Figure 6: Cost-complexity pruning curve.

9.4 Random forest

The random forest averages many decorrelated trees. We vary the number of trees and the minimum leaf size, keep the features per split at `"sqrt"`, and leave the depth unconstrained because the leaf size already limits the trees. Every configuration with leaf size 3 scores around 121.5 RMSE, while leaf size 1 has nearly 129. Doubling the number of trees from 200 to 400 changes the score by a fraction of an RMSE unit, so we keep the version with 400 trees and leaf size 3 (in favor of model complexity).

10 Experiment: Regression on log(price)

Since the target is heavily skewed (as we earlier observed), we test whether training on a transformed target helps the linear models. A handful of tickets cost more than USD 8,000 while the median is USD 234, so with squared error those rows will have an outsized weight.

We wrap the polynomial Ridge in a transformed-target regressor (`func=log1p`, `inverse_func=expm1`), which fits the model on relative errors and converts the predictions back to dollars before scoring. Least-squares on the log scale is equivalent to least-squares on the log of a multiplicative error and is far less affected by the extreme values than OLS on the raw scale. We try this against degrees 1 and 2 with $\lambda \in \{0.1, 1, 10, 100\}$ and re-evaluate the best performer according to the R^2 scores. [1]

The best configuration is degree 2 with $\lambda = 10$, at about 125.0 RMSE and $R^2 = 0.6824$, against 136.7 for the degree-2 Ridge on the raw price. Even after this, the random forest (about 121.4 RMSE) stays ahead, so this regression model (log-price Ridge) is our strongest linear model as of yet.

11 Experiment: Ensemble of Random Forest and Polynomial Regression with Ridge Regularization

Since we have two very different models performing pretty well as of now, we try testing whether averaging them helps. Two combinations are tried with `VotingRegressor`: the random forest together with the log-price Ridge, and random forest + log-price Ridge + plain Ridge.

The first combination scores about 120.0 RMSE, slightly better than random forest alone at 121.4, while the second variation scores about 125.7. Since the gap is fairly low, in favour of lesser complexity, we keep the single random forest model as the best.

12 Experiment: Testing the Random Forest with More Trees and More Features per Split

In the earlier sections, we saw that the random forest with 400 trees and max_features='sqrt' performed the best. Here we test the same model with more trees and with max_features raised to 0.5 and 0.3, so

[1] C. Tofallis, "A better measure of relative prediction accuracy for model selection and model estimation", *Journal of the Operational Research Society*, 66(8):1352–1362, 2015, §3.1 and §3.3.

each split can consider more columns. Along with this we add the count features and the duration ratio, and re-check the log-price Ridge on the new columns. 5-fold cross-validation is also carried out here like for the previous model evaluations.

Candidate	RMSE	MAE	R^2
Random forest (1000, 50% features, leaf 3, extended, with id)	117.10	42.59	0.7209
Random forest (1000, 50% features, leaf 3, extended)	118.15	43.39	0.7160
Random forest (800, 30% features, leaf 2, extended)	119.63	44.78	0.7090
Random forest (1000, sqrt, leaf 3, extended)	120.26	49.90	0.7058
Random forest (400, sqrt, leaf 3, extended)	120.33	49.91	0.7054
Random forest (400, sqrt, leaf 3, base features)	121.37	50.97	0.7003
Log-price Ridge (deg 2, $\lambda = 10$, extended)	127.00	54.98	0.6718

Table 3: Candidates in the second tuning round.

With the variations described above, we could observe that raising max_features to 0.5 with 1000 trees improves the score the most. The RMSE falls from 121.37 to 118.15 and the MAE from 51.0 to 43.4, so the model also tracks typical fares much better. The count features on their own give only a small improvement (121.37 to 120.33), and doubling the trees without changing anything else changes nothing measurable (120.26).

We can conclude that the gains have saturated at this point. Since the log-price Ridge does worse once the count columns are added, the random forest remains the stronger choice against the linear family. The row "id" is not used as a feature anywhere else, but it has some correlation with the price, that is, a slow decline across the dataset. Adding it as an extra column gives another small improvement. This achieves 117.1 RMSE and R^2 0.721, and its public leaderboard score on Kaggle is 0.76352.

12.1 An 80/20 test of the the above models

We test the above modified random forest on the 20% of the training rows kept aside, and also refit it on a much smaller subset to see how much it depends on the training volume:

Model	Training rows	RMSE	MAE	R^2
Random forest with id (1000, 50% features, leaf 3)	80%	91.79	41.33	0.7902
Random forest with id (1000, 50% features, leaf 3)	20%	105.47	54.75	0.7230
Random forest (1000, 50% features, leaf 3)	80%	93.53	42.32	0.7822
Random forest (1000, 50% features, leaf 3)	20%	104.42	54.26	0.7285
Log-price Ridge (deg 2, $\lambda = 10$)	80%	110.00	53.72	0.6987
Log-price Ridge (deg 2, $\lambda = 10$)	20%	109.00	58.44	0.7042

Table 4: Scores on the unseen 20% of the training data.

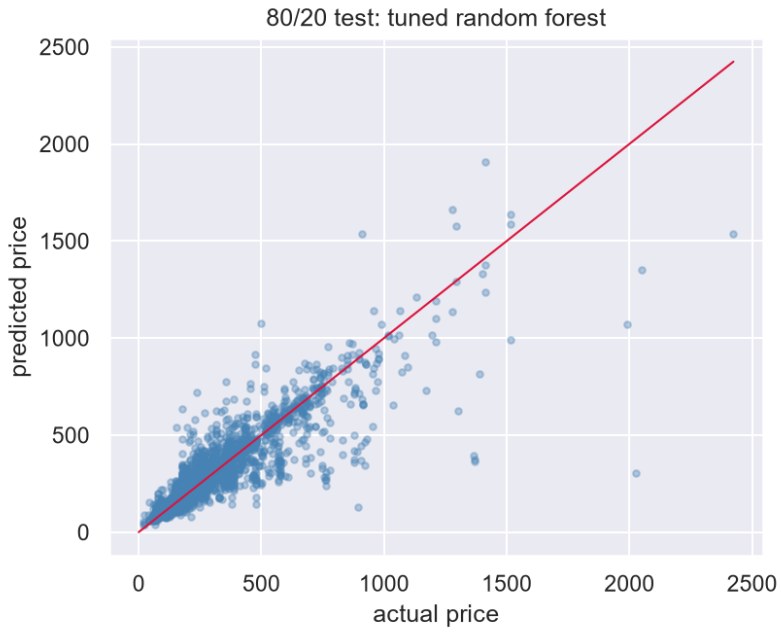


Figure 7: The tuned random forest on the unseen 20% of the training data.

When trained on 80% of the rows the random forest reaches R^2 0.782 on the unseen 20%, against 0.729 when trained on only 20% of the rows, so it still benefits from data volume. Adding the id column improves the score again, to R^2 0.790.

13 Experiment: Ensemble of HistGradientBoosting + Random Forest + ExtraTrees

In parallel with the tuning above, we also tested a combination / blended ensemble with another set of 21 engineered features (eight numeric columns, eight label-encoded categoricals and five frequency counts over composite keys). Three tree models were trained on it: histogram-based gradient boosting (500 iterations, 63 leaves), a random forest (200 trees, half the features per split) and an extra-trees model (300 trees, 70% of the features per split). Each was scored with the same 5-fold cross-validation.

Model	CV RMSE	CV R^2	Public LB
Histogram gradient boosting (500 iterations)	122.32	0.6963	
Random forest (200 trees, 50% features)	122.85	0.6937	
Extra trees (300 trees, 70% features)	124.05	0.6876	
Blend (weights 0.54 / 0.30 / 0.17)	120.82	0.7037	0.76298
Blend, cleaned rows and id column	119.38	0.7105	0.76320

Table 5: The blended ensemble on the compact feature frame.

We could observe that this combination of models improved on each of its members (120.8 against 122.3 RMSE), producing our best public leaderboard score on Kaggle, 0.78058.

14 Model diagnostics

14.1 Predicted vs actual (out-of-fold)

We collect out-of-fold predictions for the random forest and the log-price Ridge and plot them against actual prices (Figure 8). In both panels most points stay close to the diagonal for ordinary fares (up to roughly USD 1,500), and both models under-predict the rare expensive tickets. The random forest predictions rarely exceed USD 2,000 even when the actual price is far higher, which is expected, since trees average leaf values, and expensive tickets are too rare to form their own leaves.



Figure 8: Out-of-fold predicted vs actual price.

14.2 Residual analysis (random forest)

We compute the residuals $e_i = y_i - \hat{y}_i$ for the random forest and plot their histogram and their values against the predictions (Figure 9). The histogram is centred close to zero (mean -0.18) with a small number of large positive residuals, which occur only where the model predicted a high price.

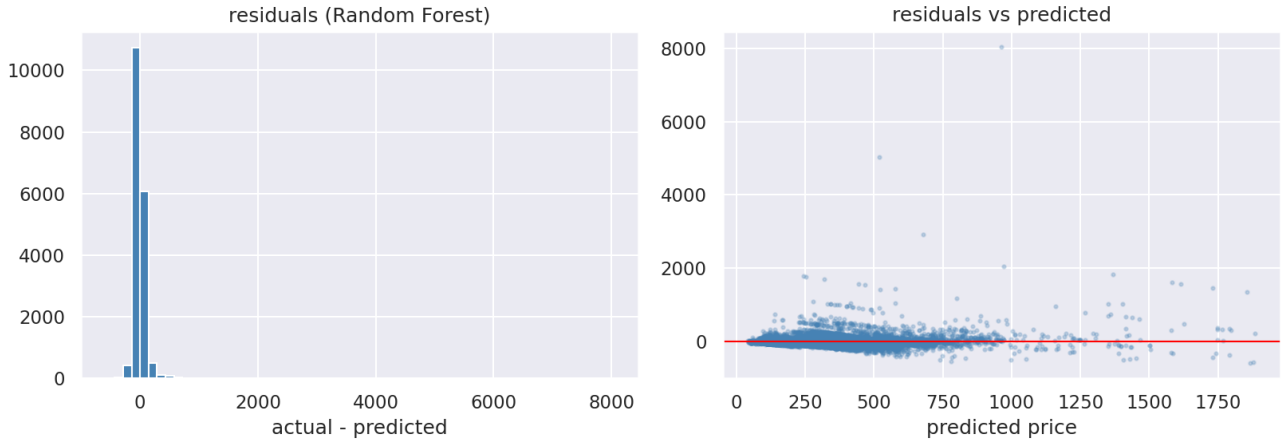


Figure 9: Random forest residuals: histogram and residuals against predictions.

14.3 Feature importance (random forest)

Finally, we refit the random forest on all training rows and draw the 15 largest impurity importances (Figure 10). The booking window, the flight length and the seating class affect the price the most, and the engineered columns are also seen to be of actual use: `flight_number` (0.05), `stops_ordinal` (0.05), the stop indicators and the Sacramento-San Jose route (0.036) all appear before the remaining categories.

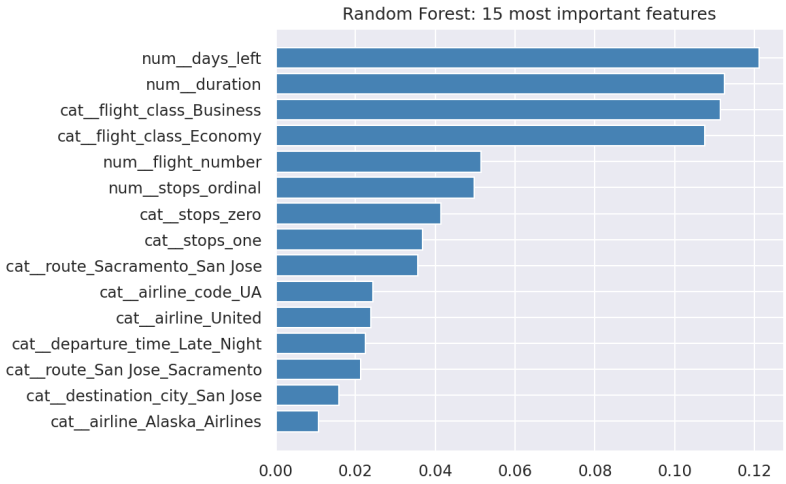


Figure 10: Random forest feature importances.

15 Cross-validation of shortlisted models

Since a single 5-fold estimate can shift with the fold split, we re-evaluate the three finalists with 5 folds repeated 3 times (15 scores each): the random forest (121.30 ± 32.18), the pre-pruned tree (130.01 ± 31.75) and the log-price Ridge (124.86 ± 30.69).

16 Metrics comparison

Model	RMSE	MAE	R^2
Random forest (400, sqrt, leaf 3)	121.37	50.97	0.7003
Log-price polynomial Ridge (deg 2, $\lambda = 10$)	124.96	55.30	0.6824
Decision tree (depth 12, leaf 5)	128.82	49.31	0.6624
Polynomial Ridge (deg 2, $\lambda = 1$)	136.67	60.86	0.6120
Decision tree, cost-complexity pruned	148.39	65.21	0.5428
Ridge (L2, $\lambda = 10$)	152.85	80.32	0.5244
Lasso (L1, $\lambda = 0.5$)	154.11	80.33	0.5168
Linear regression (OLS)	154.82	80.42	0.5107
Polynomial regression (deg 2, no penalty)	177.25	62.56	0.2742

Table 6: Flight price: full metrics comparison.

From the table, the random forest is best on RMSE and R^2 (121.4, 0.70) and second on MAE (51.0), behind the pre-pruned tree (49.3); the log-price polynomial Ridge is the best linear-family model (125.0, 0.68), while the plain polynomial and unregularised variants are the weakest.

We can observe via the MAE values that the tree-based models track typical fares well but end up having decreased performance due to for larger errors on expensive tickets.

17 Submission CSV Creation and Best Model Choice

We fit every model on all training rows and write one CSV per model.

Table 7: Best model and its submission.

Final model	CV score	Public LB	Submission file
Random forest, 1000 trees, 50% features, leaf 3, with id	R^2 0.7209	0.76352	`submission.csv`

The public scores of the various models we tested were: log-price Ridge 0.70155, pre-pruned tree 0.69389, pruned tree 0.64657, polynomial regression 0.63352, Ridge 0.55503, Lasso 0.54922, OLS 0.53131, and the forest plus Ridge ensemble 0.74533; the tuned forest and its variant having the id column were 0.75904 and 0.76352 and we achieved the best public score with the ensemble of HistGradientBoosting + Random Forest + ExtraTrees (best model).

18 Conclusion

- The target is heavily skewed, so the squared error is sensitive to the rare expensive tickets and the cross-validation scores are spread widely.
- Cleaning out data errors (like the 25 flights priced at zero) and building custom features helped increase performance, as seen in the feature importance of the airline code, flight number, exact route and number of stops for the random forest models.
- Our basic degree-3 polynomial completely overfit the data; decreasing to degree 2 and adding a small L2 penalty (Ridge) gave us our best linear model at 136.7 RMSE. With the Lasso penalty as well, we could match that performance (and it ignored about a third of our features, keeping the model less complex).
- When tuning the decision trees, setting up limits before training (pre-pruning) worked much better than trimming the tree after it grew (128.8 against 148 RMSE).
- Transforming our target to predict the log of the price increased the performance of the linear models manifold. We tried averaging that log-price Ridge model with the random forest, which nudged the overall RMSE down to 120.0, but it was only a marginal gain for increased complexity, so we did not go forward with that.
- We also tried the random forest with more trees and max_features at 0.5 together with the new count features. That brought the RMSE down to 118.2 and the MAE to 43. A blended ensemble of boosting and tree models scored 0.78058, hence becoming our best model.

Machine Learning Course Project-I

Industrial Machine Condition Classification

Team Name: Carrot

Name	Roll number	Contribution
Aradhya Chakrabarti	MT2026012	Preprocessing for the classification competition (data loading, cleaning and feature engineering); Models Tested: logistic regression (linear and quadratic), LDA, QDA, Gaussian naive Bayes, decision trees, random forests (with cross-validation, evaluation and hyperparameter tuning). Achieved the team's best public leaderboard score for the machine condition classification competition
Ankit Lamba	MT2026122	Exploratory testing and data analysis for the flight competition; tested SVR, linear regression and random forest baselines
Kunal Yadav	MT2026047	Ran preliminary prediction experiments for the flight competition
Kshitij Beri	MT2026046	Implemented an ensemble model (blend) of trees, forests and boosting; achieved the team's best public leaderboard score on the flight competition

Source code

The full code for this competition is in the project repository: `https://github.com/TimeATronics/AIT511_Project1`

1 Problem and dataset

We are given a classification problem: classify a machine into one of three states using sensor measurements, with 0 for normal, 1 for thermal failure and 2 for mechanical failure. Thermal failures are driven by temperature and cooling conditions, and mechanical failures by load, wear, torque and vibration.

The training file `student_dataset.csv` has 16,000 rows, and the test file has 4,000 rows. The columns are unit_type (S/M/H) and ten sensors: ambient_temp, process_temp, coolant_flow, shaft_speed, drive_torque, power_load, component_wear, thermal_stress, mechanical_load and vibration_index. The column unit_id identifies the machine (16,000 unique values, none of them in the test set) and is never used as a feature. The evaluation metric is accuracy.

We have used the following models: logistic regression (linear and quadratic basis), LDA, QDA, Gaussian naive Bayes, decision trees, and random forests, along with ensembles comprising of them and variations of the same.

2 Data loading

We load the three CSV files (training, test and the sample submission) with pandas. There are no duplicate rows, but every column has a small number of missing sensor readings, so we handle them during preprocessing and remove no rows.

3 Exploratory data analysis

3.1 Class distribution

For EDA, the first thing we check is the class balance, since an uneven split can make the accuracy higher than what the model is actually capable of. A count block and a bar plot show the classes are 8,000 / 4,000 / 4,000, that is a 2:1:1 split. A model that always predicts "normal" would already score

50% accuracy, so we report balanced accuracy and metrics per class alongside accuracy later on while evaluating models.

3.2 Sensor analysis by class

Next we look at how the sensors behave for each class, since the failures are supposed to reflect the physics, that is, thermal problems should show up in temperature and stress variables, mechanical problems in wear, load and vibration. The code computes the sensor means per class and draws bar graph of means and boxplots (Figures 1 and 2). We check the average values per class below to see if that information can be observed in the data.

- Thermal failures (class 1) are hotter: mean process_temp 72.4 against 54.7 for normal and 62.2 for mechanical, and mean thermal_stress 37.9 against 21.3 for normal.
- Mechanical failures (class 2) show more wear and load: mean component_wear 165.0 against 87.4 for normal, mechanical_load 88.0 against 64.8, and vibration_index 2.10 against 1.51.
- Normal machines (class 0) have the lowest values of these variables, and the boxes for the two failure classes overlap the least exactly where the physical readings differ the most.



Figure 1: Mean sensor readings per class for six key sensors.

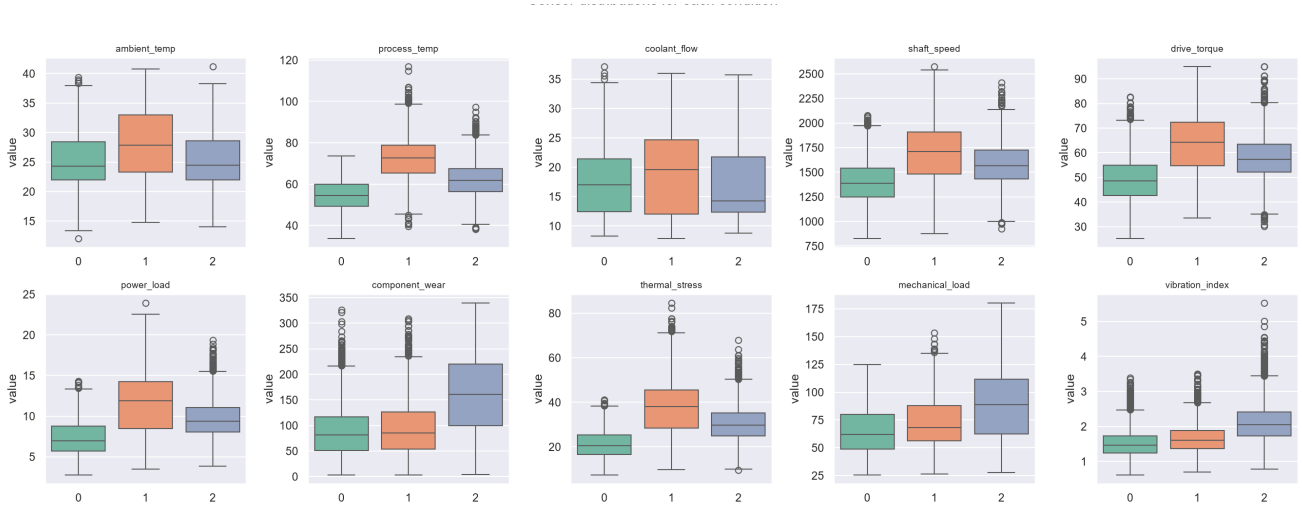


Figure 2: Every sensor by class.

3.3 Missing value analysis

We then check how many values are missing in each column, in order to decide how to fill them. The column with the most missing values is vibration_index, which has 0.46% of its values missing (74 rows),

and the others fall between 0.14% and 0.38%. With numbers this small, filling the gaps with the column median is safe and no rows need to be dropped. unit_type has 22 missing entries (0.14%), which are filled with the most common type.

3.4 Correlation analysis

Next we check the correlations between the sensors, because they decide whether models that assume independent features are suitable. The heatmap block prints the strongest pairs as well. Fifteen pairs correlate above 0.5: the strongest are power_load with thermal_stress (0.93), drive_torque with power_load (0.92), and process_temp with thermal_stress (0.86). The sensors are therefore not really independent, which matters for the Naive Bayes classifier later.

3.5 Machine type analysis

Finally we check the machine type, since failure rates may depend on it. We observe that it is not neutral. 42.4% of H machines are thermal failures, while S machines lean towards mechanical failures (29.4%), so we retain type as a one-hot feature.

4 Data cleaning

There are no impossible values and no duplicate rows, so no rows are removed.

5 Feature engineering

Since the EDA showed that the two failure classes differ mainly in temperature and in wear and load, we add two kinds of extra features:

1. `temp_difference` = process_temp − ambient_temp. Thermal failures are driven by the machine being hotter than its surroundings.
2. `power_per_speed` = power_load / shaft_speed, and `torque_per_wear` = drive_torque / component_wear. For cases where division by zero occurs, the imputer handles those as missing values.

6 Data preprocessing

All preprocessing is done with a scikit-learn `ColumnTransformer` and used inside a `Pipeline`, so during cross-validation the transformations are fitted on the training folds only.

- Missing values: the median for numeric columns, the most frequent value for unit_type.
- Encoding: unit_type is one-hot encoded, with `handle_unknown="ignore"` in case the test set contains a type that training did not see.
- Scaling: standardisation for the models that need it (logistic regression, LDA, QDA, naive Bayes).

7 Model evaluation

The metric of the competition is accuracy, but as noted above, accuracy on its own can hide problems when the classes are uneven, so balanced accuracy and macro F_1 are recorded as well. For a two-class problem with predictions $\hat{y}$ and labels y,

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \qquad F_1 = \frac{2\,\text{precision}\,\text{recall}}{\text{precision} + \text{recall}},$$

and for the multi-class versions of these metrics, we can average these per-class values (macro F_1). Balanced accuracy is the average of the per-class recalls. It treats the three classes as equally important even when the split is 2:1:1. The ROC curve plots

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{against} \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}},$$

and the area under it (AUC) summarises the ranking quality across all thresholds.

For validation we split the training data into $K = 5$ folds (maintaining the balance), and average the five scores: $\frac{1}{K}\sum_{k=1}^{K} e_k$. The final shortlisted models are re-checked with 5 folds repeated 3 times (15 scores each). A fixed random state (42) is used for every fold split and every model, to maintain reproducibility.

8 Model family direct comparisons (baseline)

We first train each model family once with reasonable settings:

- **LDA**
- **QDA**
- **Gaussian naive Bayes**: independent Gaussian features per class, $p(x \mid y) = \prod_{j=1}^{d} p(x_j \mid y)$.
- **Logistic regression**: $\eta(x) = \sigma(w^\top x) = 1/(1 + e^{-w^\top x})$.
- **Decision tree**: Recursive binary splitting with the Gini impurity, $\text{Gini}(S) = 1 - \sum_k p_k^2$.
- **Random forest**: A majority vote across many decorrelated trees.

See Figure 3.

Model	CV accuracy
Random forest (300)	0.8776
QDA (raw sensors)	0.8476
Logistic regression (linear)	0.8280
LDA	0.8239
Decision tree (unrestricted)	0.8056
Gaussian naive Bayes	0.7472

Table 1: Baseline comparison of the model families.

We can observe that the random forest leads at 0.878, and QDA is the strongest of the three Gaussian models at 0.848. Plain logistic regression (0.828) and LDA (0.824) are below QDA because their boundaries are linear, and Gaussian naive Bayes is last at 0.747 because it assumes independent sensors while several pairs are correlated.

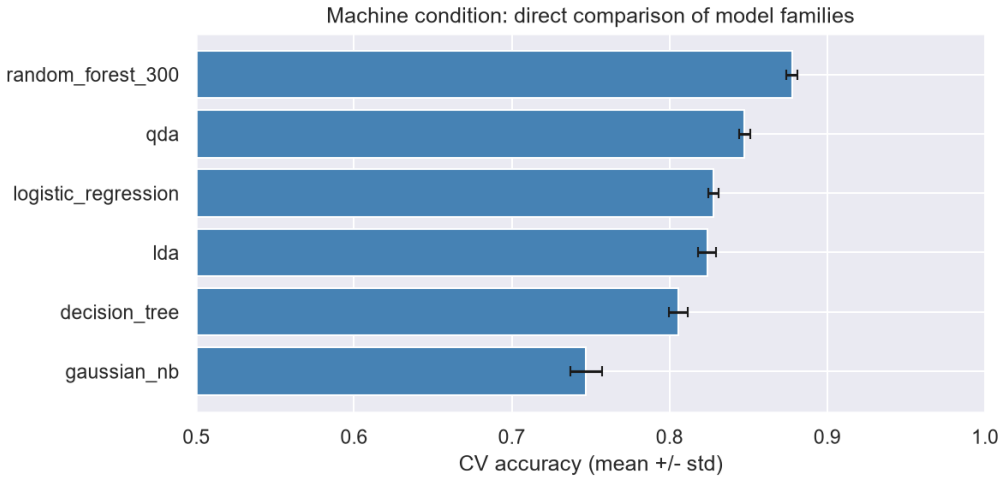


Figure 3: Direct comparison of the model families (CV accuracy).

9 Hyper-Parameter Tuning

9.1 Logistic Regression (Linear): Regularisation Strength

The strength is set through `C`, the inverse of the L2 penalty, and we iterate over it. The results are: $C = 0.01$ gives accuracy 0.818, $C = 0.1$ gives 0.825, $C = 1$ gives 0.828, $C = 10$ gives 0.829 and $C = 100$ gives 0.829. Accuracy rises up to $C = 10$ and stays flat. Hence we can see that a weaker penalty coefficient is better.

9.2 Logistic Regression with a Quadratic Basis

Since the boundaries in this dataset are not linear, we give the linear model squared and interaction terms of the sensors. This provides a quadratic boundary, using a pipeline with `PolynomialFeatures` at degree 2 before the logistic model. We try degree 1 versus 2 and $C \in \{0.01, 0.1, 1, 10\}$. Degree 2 with $C = 1$ reaches 0.8853, clearly above the linear version (0.829).

9.3 QDA: Covariance Regularisation

A small regularisation is added to each covariance, which keeps them invertible when the features are strongly related (singular matrices). When we do not add a regularization parameter, that is, `reg_param` = 0 the code crashes because `temp_difference` is an exact linear combination of two sensors and the class covariance is singular. With regularisation, the QDA model shows accuracy of 0.8393, but it is below the previous accuracy of 0.8476 obtained from the raw sensors at `reg_param` = 0.1. Hence we continue using the raw-sensor version for QDA.

9.4 Gaussian Naive Bayes: Variance Smoothing

The Naive Bayes classifier assumes the sensors are conditionally independent given the class and adds a small smoothing value to every variance. We iterate over the smoothing value from 10^{-9} to 0.1. The best setting (0.1) increases the score from 0.747 to 0.754 (still far lower than the rest).

9.5 Decision Tree: Pre-pruning and Post-pruning

For pre-pruning we tune four early stopping criteria: maximum depth, minimum leaf size, minimum samples needed to split a node, and the maximum number of leaves. The best pre-pruned tree has depth 12 with a minimum leaf size of 5 and scores 0.8389. The unrestricted tree falls to 0.806, and depth-4 trees underfit around 0.772. For post-pruning we use the cost-complexity criterion

$$R_\lambda(T) = R(T) + \lambda|T|,$$

with candidate λ values from the pruning path. Here post-pruning helps: the best λ (about 3×10^{-4}, keeping 161 leaves) increases the cross-validation accuracy of the tree from 0.8389 to 0.8429 (Figure 4), while larger λ values remove useful branches.

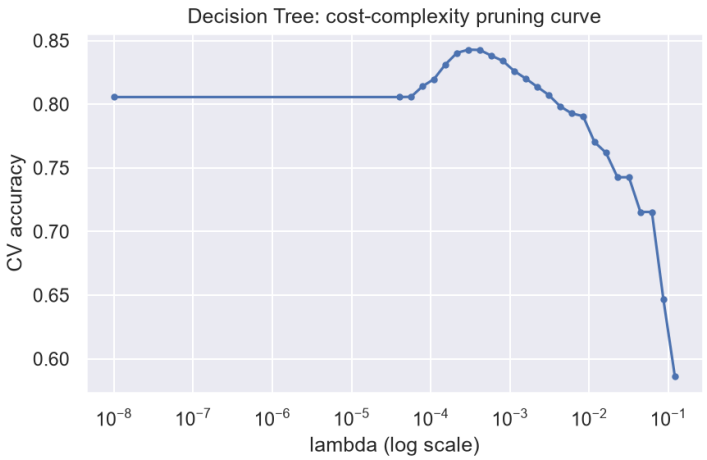


Figure 4: Cost-complexity pruning curve for the classification tree.

9.6 Random forest

We tune the depths per tree, the minimum leaf size and the class weighting. We include class weights because the classes are split 2:1:1. The unweighted depth-14 random forest is the best at 0.8787, but the differences inside the unweighted block are tiny (0.876 to 0.879). Every `balanced` variant scores lower than the corresponding unweighted versions.

10 Experiment: Class Imbalance Handling

Since the 2:1:1 split could in principle hurt the two failure classes even while accuracy looks fine, we compare three approaches further on the two best models seen so far (the quadratic logistic regression and QDA): (i) reweighting the classes, (ii) random oversampling of the minority classes, and (iii) SMOTE, which creates synthetic minority samples by interpolating between neighbours.

Table 2: Imbalance strategies for the quadratic logistic (5-fold CV).

Variant	Accuracy	Macro F_1
none	0.8853	0.8756
reweight (class weights)	0.8743	0.8666
random oversampling	0.8728	0.8649
SMOTE	0.8734	0.8660

Each of these experiment scenarios lowered both metrics. With classes that are already well separated and only a 2:1:1 split, the resampled training set biases the boundary towards synthetic or duplicated minority points without improving any real case, so we continue using the previous models only.

11 Experiment: Combining/Blending the Top Models

The two strongest single models seen so far are the quadratic and the cubic logistic regressors. We further tried using a cubic basis with stronger regularisation (degree 3, $C = 0.3$), which reached an accuracy of 0.8894 ± 0.0031 under cross-validation, just above the quadratic model. We also tested three soft-voting combinations of the pair (considering a combination with the random forest in the third variant): The three variants are (i) an even 50/50 vote, (ii) a 60/40 vote weighted toward the quadratic model, and (iii) a 40/40/20 vote with the random forest. Each combination is scored with the same 5-fold cross-validation as everything else before.

Combination	Accuracy	Balanced acc.	Macro F_1
Quadratic 40% + cubic 40% + forest 20%	0.8919	0.8746	0.8828
Quadratic 50% + cubic 50%	0.8909	0.8738	0.8819
Quadratic 60% + cubic 40%	0.8901	0.8730	0.8810

Table 3: The three probability-vote combinations under 5-fold cross-validation.

All of these three combinations performed better than all previous models used by us so far. As per the public leaderboard scores, the 60/40 vote performs best at 0.89125, followed by the 40/40/20 vote (0.89062) and the even vote (0.89000), hence it is our best model.

12 An 80/20 test of the best model

We also test the best model and its strongest single-model member on an 80/20 split of the training data. This shows us how the models behave on rows they have never seen, and how much they still depend on the amount of training data.

Model	Training rows	Accuracy	Balanced acc.	Macro F_1
Soft vote (60/40)	80%	0.9003	0.8854	0.8921
Soft vote (60/40)	20%	0.8928	0.8763	0.8834
Quadratic logistic	80%	0.8953	0.8798	0.8862
Quadratic logistic	20%	0.8872	0.8731	0.8782

Table 4: Scores of the best model and the quadratic logistic on the unseen 20% of the training data.

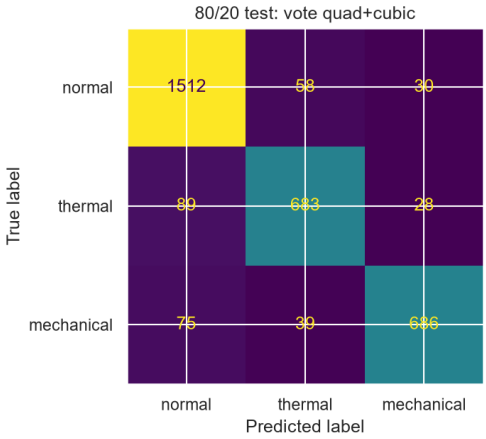


Figure 5: Confusion matrix of the best model on the unseen 20% of the training data.

When we train the ensemble model on 80% of the rows it scores 0.9003 on the 20% remaining, and still 0.8928 from just 20%, so the decision boundary is stable and does not depend upon data volume; The quadratic logistic model also has similar, but slightly lower metrics.

13 Model Diagnostics

Confusion matrices. We compute out-of-fold predictions for the three leading models and draw their confusion matrices (Figure 6), with labels 0 normal, 1 thermal, 2 mechanical. With all the models, normal machines are recognised best (7,498 of 8,000 for the quadratic logistic). The two failure classes each have between 660 and 670 misclassifications, and most of those misses fall into the normal class, roughly 470 to 480 per class. Only about 180 to 200 machines, upon classification, get swapped between thermal and mechanical, and those two classes are the ones whose sensor boxes overlap the most.

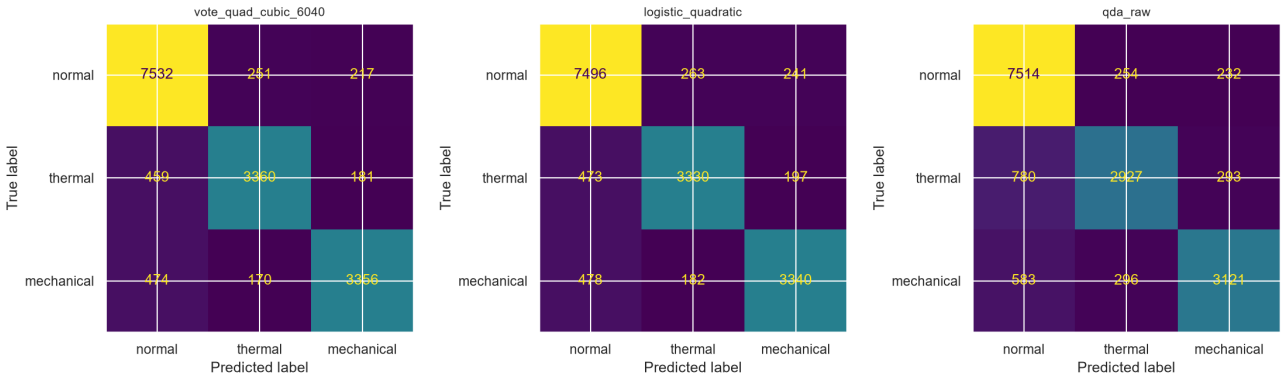
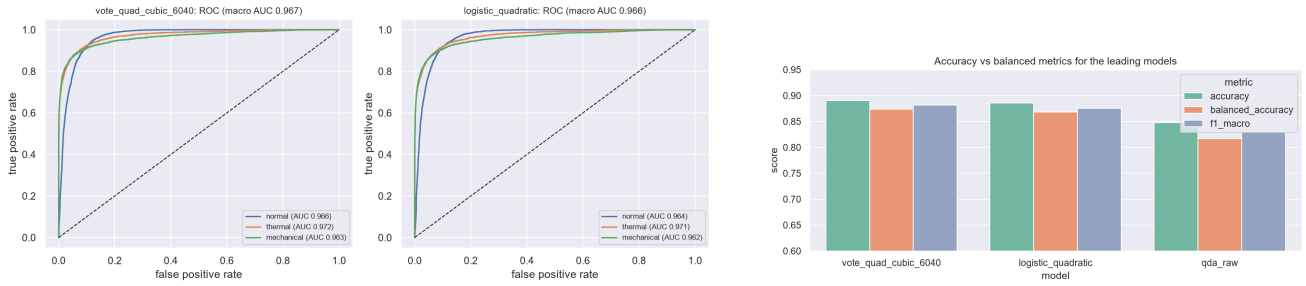


Figure 6: Out-of-fold confusion matrices for the three leading models.

Per-class metrics. We then print the full classification report for the quadratic logistic regression. Recall is 93.7% on normal, 83.3% on thermal and 83.5% on mechanical, with precision and F_1 following closely. No class collapses, and the weakest recall is still above 80%.

ROC curves. Next we draw one-vs-rest ROC curves from the out-of-fold class probabilities and print the area under each curve. The two have almost identical overall discrimination (macro AUC 0.967 for the 60/40 vote against 0.966 for the quadratic logistic). The curves tend to go towards the top-left corner, and in both cases the thermal class has the highest AUC (0.972 for the 60/40 vote, 0.971 for the quadratic logistic).

Balanced metrics. Finally we compute accuracy, balanced accuracy and macro F_1 for the three models and plot them side by side (Figure 7). The 60/40 vote-based ensemble model is best on all three metrics (0.8905, 0.8735 and 0.8814), the quadratic logistic model follows (0.8854, 0.8682 and 0.8757) and QDA is the worst out of these three.



(a) one-vs-rest ROC

(b) accuracy vs balanced metrics

Figure 7: Discrimination and class-balance checks.

t-SNE. We also draw a t-SNE projection of a 3,000-row sample, before and after feature engineering (Figure 8), as an illustration of how the classes are present relative to each other. On the raw sensors the three classes form broad overlapping clouds, with mechanical machines mostly in the upper region and thermal machines largely on the right. After adding the engineered features the thermal cloud becomes smaller on the right and the mechanical group separates more cleanly from the normal machines in the middle.

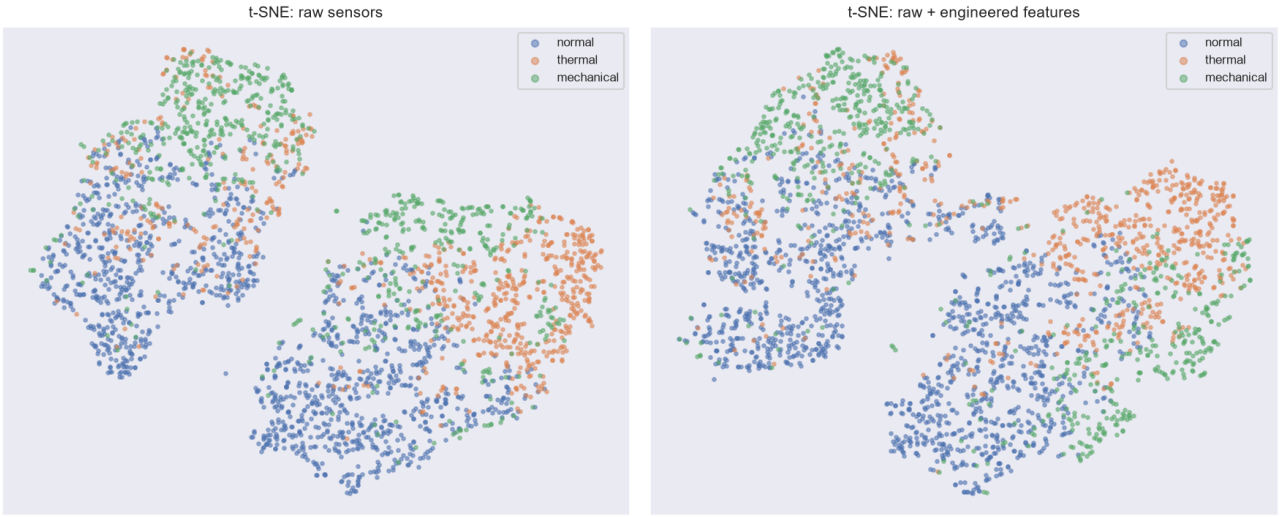


Figure 8: t-SNE before and after feature engineering.

14 Cross-validation of shortlisted models

Since a single 5-fold estimate can change with the fold split chosen, we re-evaluate the three best models with 5 folds repeated 3 times: the quadratic logistic regression (0.8853 ± 0.0056), the random forest (0.8777 ± 0.0051) and QDA (0.8476 ± 0.0068). We checked the same for the ensembled model (60/40 voting based) in the 80/20 test above.

15 Metrics comparison

We finally evaluate one representative configuration per family under accuracy, balanced accuracy and macro F_1.

Model	Accuracy	Balanced acc.	Macro F_1
Logistic regression, quadratic basis (deg 2, $C = 1$)	0.8853	0.8681	0.8756
Random forest (depth 14)	0.8787	0.8554	0.8655
QDA (raw sensors, reg 0.1)	0.8476	0.8171	0.8299
Decision tree, cost-complexity pruned	0.8429	0.8194	0.8259
Decision tree, pre-pruned (depth 12, leaf 5)	0.8389	0.8126	0.8206
Logistic regression, linear ($C = 10$)	0.8293	0.8037	0.8104
LDA	0.8239	0.7893	0.8022
Gaussian naive Bayes (smoothing 0.1)	0.7535	0.7244	0.7273

Table 5: Machine condition: full metrics comparison.

From the table above, we see that the quadratic logistic regression leads on all three metrics among the single models, the random forest follows closely, the post-pruned tree is the best tree, and Gaussian Naive Bayes performed the worst. The ensemble model based on the 60/40 voting is ahead of every other model (0.8901 accuracy).

16 Submission CSV Creation and Best Model Choice

For the submissions, we fit every model on all training rows and write one CSV per model.

Table 6: Best model and its submission.

Final model	CV score	Public LB	Submission file
Soft vote of quadratic (60%) and cubic (40%) logistic	acc 0.8901	0.89125	`submission.csv`

The public leaderboard scores on Kaggle were: quadratic logistic 0.88812, cubic logistic 0.88687, random forest 0.88125, QDA 0.85625 and the post-pruned tree 0.84937. We checked the scores for the ensemble models in previous sections (higher than all of these).

17 Conclusion

- Even though the failure types are unevenly distributed (a 2:1:1 split), our accuracy metric isn't masking poor performance on a minority class. We could observe that seeing the lowest recall was 83.3% for thermal failures, and our top model also led on accuracy, balanced accuracy and macro F_1.
- The raw sensor data separates the failure types in the same way that they logically and physically should, and our custom engineered features made the boundaries between them definite.
- The decision boundaries between the machine states are curved. Hence, upgrading standard logistic regression with degree-2 polynomial features increased the accuracy from 0.829 to 0.885, better than both QDA (0.848) and the random forest (0.879); a cubic basis with stronger regularisation reached even higher accuracy (0.889).
- When tuning the decision trees, we tested both pruning methods: setting limits beforehand (pre) (max depth 12, minimum leaf size 5) gave a maximum of 0.839 accuracy, while cost-complexity (post-)pruning increased that further to 0.842.
- We tried handling the class imbalance with class reweighting, oversampling and SMOTE, but all of them were detrimental to the performance metrics.

- Finally, the best-performing model was the 60/40 combination (with soft voting) of the quadratic and cubic logistic regressions: 0.8901 accuracy in cross-validation, and 0.89125 on the public leaderboard for the industrial machine condition classification competition on Kaggle.